Software Architecture Overview

Project: Skill/Interest-Based Secure Matching Platform for Erasmus and Local Students

Section author: Alberto Alvarez Polo – to be integrated into the Software Requirements Specification (PA1)

This section describes the high-level architecture of the platform and how it supports the four core use cases (UC1–UC4).

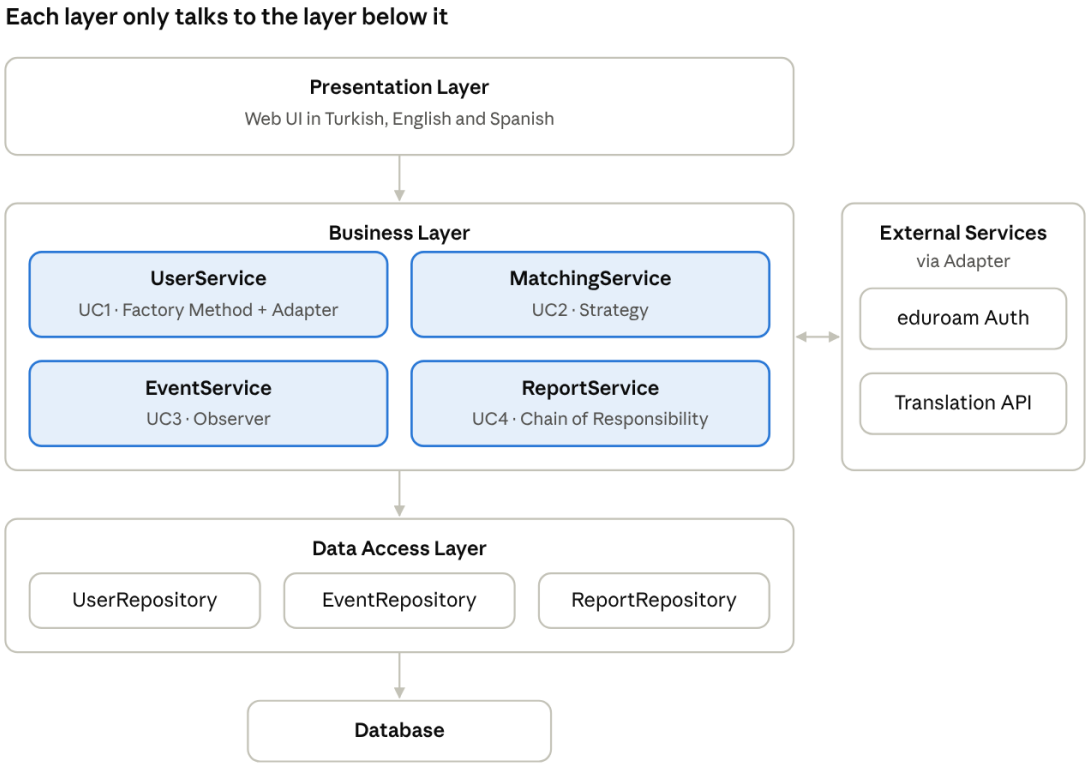
1. Architectural Style: Layered Architecture

The platform uses a **Layered Architecture** with three internal layers (Presentation, Business and Data Access) and a set of external services connected through Adapters. Each layer only communicates with the layer directly below it.

We chose this style for three reasons:

- **Separation of concerns:** user interface, business rules and data storage are kept apart, so a change in one layer does not break the others.
- **Maintainability and testability:** each service can be developed and tested independently, which suits a three-person team working in parallel.
- **Natural fit for design patterns:** the Business Layer is where Factory Method, Strategy, Observer and Chain of Responsibility live, and Adapters isolate the system from external services such as eduroam and the translation API.

2. Architecture Diagram



layered architecture · 3 layers + external services

Requests flow from the Web UI down to the Business Layer services and from there to the repositories and the database; external services are reached only through Adapters.

3. Layer Descriptions

Layer	Components	Responsibility
Presentation	Web UI (Turkish / English / Spanish)	Displays screens, collects user input and shows results and notifications
Business	UserService, MatchingService, EventService, ReportService	Implements the business rules of each use case and applies the design patterns
Data Access	UserRepository, EventRepository, ReportRepository, Database	Stores and retrieves users, profiles, events, matches and reports
External Services	eduroam Authentication, Translation API (via Adapter)	Third-party systems wrapped by Adapters so the Business Layer

Layer	Components	Responsibility
		does not depend on their interfaces

4. How the Architecture Supports the Use Cases

Each use case has its own service in the Business Layer, so the four use cases can be designed, implemented and tested independently.

Use Case	Layers involved	Pattern applied
UC1 Register & Create Profile	Web UI → UserService → UserRepository; eduroam Adapter for authentication	Factory Method, Adapter
UC2 Find a Match	Web UI → MatchingService → UserRepository	Strategy
UC3 Create/Join Event & Get Notified	Web UI → EventService → EventRepository; notifications sent back to subscribed users	Observer
UC4 Report Inappropriate Content	Web UI → ReportService → ReportRepository	Chain of Responsibility

- **UC1:** the UserService uses a Factory Method to create the right user type (Erasmus or local) and an Adapter to verify the user through eduroam without depending on its interface.
- **UC2:** the MatchingService delegates the matching logic to an interchangeable Strategy (language exchange, hobbies, sports), so new criteria can be added without changing the service.
- **UC3:** the EventService acts as the subject of an Observer, notifying every subscribed user when a match or event changes.
- **UC4:** the ReportService passes each report along a chain of handlers (automatic filter → moderator → admin) until one of them resolves it.